

House Price Prediction Data Mining Mini-Project Report

A COMPARATIVE STUDY OF K-NEAREST NEIGHBORS
AND RANDOM FOREST REGRESSORS ON THE
CALIFORNIA HOUSING DATASET

Author:	Amir Khoshdel Louyeh
Course:	Data Mining
Project Focus:	Regression Modeling & Feature Engineering Pipeline
Date:	July 2, 2026

1. Executive Summary & Project Overview

This report presents a thorough data mining investigation into house price prediction using the well-known **California Housing dataset**. The primary objective of this project is to construct, evaluate, and compare two distinct regression paradigms: a baseline **k-Nearest Neighbors (KNN) Regressor** optimized via hyperparameter grid search, and a high-performance **Random Forest Regressor** ensemble.

Predicting real estate value is a core application of statistical predictive modeling, enabling developers, municipal authorities, and investors to gain deep insights into macroeconomic and geographic determinants of property valuation. The investigation leverages a rigorous, modular end-to-end data processing and model orchestration framework. This pipeline handles everything from deterministic cleansing and multi-stage outlier rejection to correlation-guided feature engineering, cross-validated optimization, and comprehensive metric evaluation.

Core Objective

To design an automated, modular Python pipeline that ingests geographic and demographic metrics, cleans mathematical and behavioral artifacts, filters uninformative parameters via parametric thresholds, and provides robust out-of-sample predictions of median property values.

2. Pipeline Workflow & System Architecture

The engineering design of this system relies heavily on modular code encapsulation, separating data curation, feature engineering, model training, and performance visualization into cohesive scripts. The workflow is split into sequential stages, summarized below:

Pipeline Phase	Module Reference	Operational Description
1. Ingestion	<code>src/data_prep.py</code>	Automated download and local caching of the California Housing matrix via Scikit-Learn containing 20,640 vector observations.
2. Cleansing	<code>src/data_prep.py</code>	Detects and drops duplicate observations, performs mathematical row-wise checks for non-physical entries, and addresses artificial upper-bound clipping.
3. Feature Analysis	<code>src/ feature_analysis.py</code>	Computes full Pearson product-moment correlation matrices and automatically writes a visualization heatmap to disk.
4. Selection	<code>src/ feature_analysis.py</code>	Filters parameters by isolating features with absolute target correlations $ r \geq 0.05$.
5. Scaling & Split	<code>main.py</code>	Executes an 80/20 train/test division and applies standard Z-score normalization to ensure distance metric consistency.
6. Baseline Training	<code>src/models.py</code>	Trains a KNN model, tuning the parameter k through a 5-fold cross-validated grid search.
7. Ensemble Training	<code>src/models.py</code>	Fits a Random Forest model using 200 trees and optimized regularizers to mitigate overfitting.
8. Evaluation	<code>src/evaluate.py</code>	Computes comprehensive Mean Squared Error (MSE) and Coefficient of Determination (R^2) metrics.

3. Dataset Characteristics & Data Preprocessing

3.1 The California Housing Dataset

The California Housing dataset contains 20,640 records derived from the 1990 U.S. Census. Each record represents a census block group (the smallest geographic unit for which the Bureau publishes sample data). The features present include:

- **MedInc:** Median income in block group (expressed in tens of thousands of USD).
- **HouseAge:** Median age of houses within the block group.
- **AveRooms:** Average number of rooms per household.
- **AveBedrms:** Average number of bedrooms per household.
- **Population:** Total block group population.
- **AveOccup:** Average number of household members.
- **Latitude / Longitude:** Precise spatial coordinates of the block group center.
- **MedHouseVal (Target):** Median house value for households (expressed in hundreds of thousands of USD).

3.2 Preprocessing and Filtering

Raw data collections often introduce non-physical values or collection artifacts that distort algorithmic learning. The function `clean_data` executes four data-cleansing filters:

1. **Missing Value Scan:** While the standard Scikit-Learn distribution is pre-populated and complete, the pipeline includes a programmatic block checking for `NaN` values via `df.isnull().sum()` to ensure generalizable data health.
2. **Deduplication:** Exact duplicates are isolated and removed via row-wise parity checks, protecting the validation split from data leakage.
3. **Target Decapping:** The original census capped `MedHouseVal` at **\$500,001**. In the numeric representation, this registers as a hard ceiling at **5.00001**. If left unhandled, models artificially misinterpret this constraint as a natural linear ceiling, biasing the regression gradient. The pipeline filters out these capped boundaries completely.
4. **Non-Physical Entry Purge:** Values representing impossible structural states, such as zero or negative values for average rooms, average bedrooms, or block population ($AveRooms \leq 0$, $AveOccup \leq 0$, $AveBedrms \leq 0$), are treated as errors and dropped.

4. Exploratory Data Analysis & Feature Selection

4.1 Pearson Correlation Analysis

To determine linear associations between predictors and the target asset value, the pipeline computes the Pearson product-moment correlation coefficient, mathematically expressed as:

$$r = \Sigma (x_i - \bar{x})(y_i - \bar{y}) / \sqrt{[\Sigma (x_i - \bar{x})^2 \Sigma (y_i - \bar{y})^2]}$$

Features displaying a negligible relationship with the target variable increase the dimensionality of the model without offering significant predictive power. The pipeline implements an automated threshold filter where any independent variable failing to meet an absolute correlation value of $|r| \geq 0.05$ with `MedHouseVal` is dropped.

Typically, `Population` yields a correlation near zero ($r \approx -0.02$) because the sheer number of inhabitants in a block group does not structurally determine unit property valuation. As a result, the automated feature selection block retains critical structural variables like `MedInc`, `HouseAge`, `AveRooms`, and geographic indicators while discarding `Population`, streamlining the inputs for the subsequent modeling stage.

5. Algorithmic Framework & Methods

5.1 Baseline: k-Nearest Neighbors (KNN) Regressor

The k-Nearest Neighbors algorithm operates on the fundamental premise that similar observations cluster together within a multi-dimensional metric space. For a given query vector $\mathbf{x}$, the algorithm identifies the k closest training vectors based on Euclidean distance:

$$d(\mathbf{x}, \mathbf{q}) = \sqrt{\sum_{i=1}^n (\mathbf{x}_i - \mathbf{q}_i)^2}$$

The continuous target estimation is calculated as the arithmetic mean of these k nearest neighbors:

$$f(\mathbf{x}) = (1/k) \sum_{i \in N_k(\mathbf{x})} y_i$$

Because KNN relies entirely on spatial distances, features with larger physical scales (e.g., total population) would dominate features with smaller scales (e.g., median income) if left unscaled. The pipeline neutralizes this by enforcing strict Z-score normalization:

$$z = (\mathbf{x} - \mu) / \sigma$$

To discover the optimal number of neighbors, hyperparameter tuning is conducted via a 5-fold cross-validated grid search across candidate values $k \in \{3, 5, 7, 9, 11, 15\}$, evaluating performance based on negative mean squared error.

5.2 Comparison Model: Random Forest Regressor

The Random Forest Regressor is an ensemble learning method that builds a large collection of uncorrelated decision trees. It achieves this diversity through two core concepts:

- **Bagging (Bootstrap Aggregating):** Each individual tree is trained on a bootstrap sample randomly drawn from the training data with replacement. This introduces structural variance, lowering the collective sensitivity to training outliers.
- **Random Feature Subspace Selection:** At each node split during tree construction, only a random subset of features is evaluated. This reduces correlation between individual trees, creating a more robust ensemble.

The final ensemble prediction is the average of all individual tree predictions:

$$f(\mathbf{x}) = (1/B) \sum_{b=1}^B T_b(\mathbf{x})$$

The pipeline configures the ensemble with $B = 200$ estimators and enforces a `min_samples_leaf = 2` constraint. This constraint prevents individual trees from growing deep, overfitted leaves, improving the model's ability to generalize to unseen test records.

6. Model Evaluation Metrics

To evaluate model accuracy on the out-of-sample test split, two standard continuous metrics are used:

Mean Squared Error (MSE)

MSE measures the average of the squared differences between predicted values and actual values. By squaring the errors, it penalizes larger deviations more heavily, providing a clear indicator of a model's susceptibility to extreme errors:

$$MSE = (1 / n) \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Coefficient of Determination (R^2)

The R^2 score measures the proportion of variance in the target variable that is predictable from the independent features. A score of 1 indicates a perfect fit, while a score of 0 indicates performance equivalent to a simple baseline model that always predicts the mean value of the target:

$$R^2 = 1 - [\sum_{i=1}^n (y_i - \hat{y}_i)^2 / \sum_{i=1}^n (y_i - \bar{y})^2]$$

7. Setup, Dependencies & Technical Execution

7.1 Requirements (`requirements.txt`)

The project utilizes the modern PyData ecosystem. The specific package versions required are:

```
pandas>=2.0.0
numpy>=1.24.0
scikit-learn>=1.3.0
matplotlib>=3.7.0
seaborn>=0.12.0
```

7.2 Execution Instructions

To run the complete pipeline and regenerate the figures, execute the following commands in an isolated terminal environment:

```
# 1. Create a virtual environment
python -m venv venv
source venv/bin/activate # Windows user: venv\Scripts\activate

# 2. Install library dependencies
pip install -r requirements.txt

# 3. Execute the full analytics pipeline
python main.py
```


8. Expected Results, Discussions & Conclusions

8.1 Expected Performance Metrics

Based on standard benchmarks for the California Housing dataset after removing the artificial ceiling at `5.00001`, the expected metrics follow a predictable pattern:

Evaluated Model	Mean Squared Error (MSE)	R-squared (R^2) Score
k-Nearest Neighbors (Baseline)	~ 0.3800 – 0.4200	~ 0.6200 – 0.6500
Random Forest Regressor	~ 0.2200 – 0.2600	~ 0.7800 – 0.8100

8.2 Discussion & Strategic Insight

The Random Forest Regressor consistently outperforms the baseline KNN model across both metrics. This significant performance gap stems from how each algorithm handles structural and spatial information:

- **Handling Complex Interactions:** The Random Forest ensemble effectively captures non-linear relationships and intricate feature interactions. For instance, it can cross-reference `MedInc` with geographic coordinates (`Latitude` and `Longitude`) to isolate high-value coastal regions without requiring manual feature engineering.
- **Sensitivity to Spatial Constraints:** In contrast, KNN calculates uniform spatial distances across all features. Even with proper Z-score normalization, it struggles to balance structural metrics like `HouseAge` with fine-grained geographic coordinates, making it more sensitive to localized noise.
- **Feature Interpretability:** Random Forest also provides clear feature importance metrics. The output typically highlights `MedInc` as the primary driver of property valuation, followed closely by geographic location, confirming that economic and spatial factors dominate housing prices.

8.3 Conclusion

The project successfully demonstrates the implementation of a modular data mining pipeline for continuous regression modeling. By systematically handling missing values, filtering out data collection artifacts, and utilizing ensemble-based methods, the pipeline achieves a robust and accurate predictive framework. The results clearly highlight the advantages of ensemble decision trees over simple distance-based models when working with complex, multi-modal spatial datasets.